SIMATS ENGINEERING
ASSESSMENT TOOL 2: Industry Problem Solving Task

COURSE NAME:	Cloud Computing and Big Data Analytics	**COURSE CODE:**	CSA1522

COURSE OUTCOME COVERED

CO4: *Analyze big data characteristics and challenges across domains including security and application aspects.* (BL4)

Dave's Taxonomy: Articulation (Level 4) **Total Marks: 40**
Question Count: 5
Duration :60 Minutes

Code of Conduct

I KEERTHANA.L-(192521062) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: _____

Assessment Tasks

Industry Problem Solving Task

1. **Retail Data Optimization Problem -** A retail chain operates both physical stores and an online platform. It generates large volumes of sales, inventory, and customer behavior data. The company wants real-time inventory tracking and demand forecasting. It also aims to deliver personalized marketing recommendations to customers. Design a big data architecture to support these requirements. Explain the tools, technologies, and analytics models you would use.

2. **Social Media Fraud Detection Problem -** A social media platform processes billions of user interactions daily. These include posts, comments, likes, and shares. The platform wants to detect fake accounts and misinformation quickly. Real-time or near real-time processing is required. Design a scalable big data solution for this use case. Explain the role of machine learning and stream processing frameworks.

3. **Government Data Platform Problem**

   It combines census, economic, and geospatial datasets. Different departments need access to shared and integrated data. The system must ensure interoperability across multiple agencies. Strict privacy and security requirements must be followed. Propose a governance and data management strategy.

4. **Banking Fraud Detection Problem**
   A bank processes millions of financial transactions every minute. It needs to detect fraudulent activities in real time. The system must analyze historical and streaming transaction data.
   Low latency and high accuracy are critical requirements. Design a big data pipeline for fraud detection. Explain the technologies and algorithms you would implement.

5. **Agriculture Smart Farming Problem**
   A smart farming system collects soil, weather, and crop data. Sensors and drones provide real-time field information. Farmers want insights to improve yield and resource usage. Data is generated from multiple distributed sources. Design a big data solution for precision agriculture. Explain how analytics can support decision-making.

Industry Problem Solving Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 – Developing	Level 1 - Beginning
Understanding of Industry Context	20%	Demonstrates strong understanding of real-world problem context	Good understanding	Basic understanding	Weak understanding
Application of Big Data Concepts	25%	Applies highly relevant big data ideas and tools	Mostly appropriate application	Partial application	Weak application
Solution Design	25%	Solution is practical, scalable, and well-reasoned	Reasonable solution	Basic solution with gaps	Unclear solution
Security / Risk Awareness	15%	Strong awareness of security and operational risks	Reasonable awareness	Limited awareness	Poor awareness
Clarity and Professionalism	15%	Well-structured and professional response	Generally clear	Moderately clear	Poorly presented

Total Marks Awarded:

1. Retail Data Optimization Problem - A retail chain operates both physical stores and an online platform. It generates large volumes of sales, inventory, and customer behavior data. The company wants real-time inventory tracking and demand forecasting. It also aims to deliver personalized marketing recommendations to customers. Design a big data architecture to support these requirements. Explain the tools, technologies, and analytics models you would use.

Introduction

A retail chain generates data from physical stores, online shopping, inventory systems, customer transactions, mobile applications, and websites. A Big Data architecture can integrate these sources and provide **real-time inventory tracking, demand forecasting, and personalized recommendations**.

Proposed Big Data Architecture

Data Sources → Data Ingestion → Data Storage → Stream/Batch Processing → Analytics & ML → Applications

1. Data Sources

- Point-of-sale (POS) systems from physical stores.
- E-commerce transactions and clickstream data.
- Inventory and warehouse management systems.
- Customer profiles, loyalty programs, and purchase history.
- Mobile applications and website browsing behavior.

2. Data Ingestion

- **Apache Kafka** can collect continuous sales, inventory, and customer events.
- **Apache NiFi** can be used to integrate data from different sources.
- Batch data can be imported using ETL tools.

3. Data Storage

- **HDFS** or cloud data lakes can store large volumes of raw data.
- **Apache Hive** can provide SQL-based analysis over large datasets.

- **NoSQL databases such as MongoDB or Cassandra** can store rapidly changing customer and inventory information.

4. Data Processing

- **Apache Spark** can process both batch and streaming data.
- Spark Streaming can identify inventory changes almost immediately.
- Historical data can be processed to identify sales trends and seasonal patterns.

5. Analytics and Machine Learning

- **Demand forecasting:** Time-series models such as ARIMA, Prophet, or LSTM can predict future product demand.
- **Customer segmentation:** K-Means clustering can group customers according to purchasing behavior.
- **Recommendation system:** Collaborative filtering or content-based recommendation can suggest relevant products.
- **Predictive analytics:** Machine learning can identify products likely to be purchased by individual customers.

6. Real-Time Inventory Tracking

When a product is purchased, the transaction is sent through Kafka. The streaming system updates the inventory database immediately. Alerts can be generated when stock reaches a minimum level.

7. Personalized Marketing

Customer browsing and purchase history can be analyzed to create personalized offers, product recommendations, and targeted advertisements.

8. Visualization

Tools such as **Power BI or Tableau** can display:

- Current inventory.
- Sales trends.
- Predicted demand.
- Customer segments.
- Recommendation performance.

Conclusion

The proposed architecture combines **Kafka, Spark, HDFS/cloud storage, NoSQL databases, and machine learning** to provide real-time and historical analytics. It helps the retailer reduce stock-outs, improve demand forecasting, optimize inventory, and provide personalized customer experiences.

2.Social Media Fraud Detection Problem - A social media platform processes billions of user interactions daily. These include posts, comments, likes, and shares. The platform wants to detect fake accounts and misinformation quickly. Real-time or near real-time processing is required. Design a scalable big data solution for this use case. Explain the role of machine learning and stream processing frameworks.

Introduction

Social media platforms generate billions of posts, comments, likes, shares, and user activities. Since fake accounts and misinformation can spread rapidly, the platform requires a **scalable real-time Big Data architecture**.

Proposed Architecture

Users → Social Media Events → Kafka → Stream Processing → ML Models → Fraud/Misinformation Detection → Alerts & Action

1. Data Sources

The system collects:

- Posts and comments.
- Likes and shares.
- Friend/follower relationships.
- Login and account activity.
- IP/device information.
- Content and URLs.

2. Stream Data Ingestion

- **Apache Kafka** acts as a distributed message broker.
- Each interaction can be represented as an event.
- Kafka partitions allow billions of events to be processed in parallel.

3. Real-Time Processing

Apache Flink or Spark Structured Streaming processes incoming events.

It can calculate:

- Number of posts per minute.
- Abnormal sharing patterns.
- Sudden increases in activity.
- User interaction frequency.
- Account behavior patterns.

4. Fake Account Detection

Machine learning can analyze features such as:

- Account age.
- Number of followers and following.
- Posting frequency.
- Similarity of posts.
- Login patterns.
- Network connections.
- Unusual interaction behavior.

Algorithms such as **Random Forest, XGBoost, Logistic Regression, and neural networks** can classify accounts as legitimate or suspicious.

5. Misinformation Detection

Natural Language Processing (NLP) can analyze:

- Text content.
- Keywords.
- Sentiment.
- Claims.
- URLs and linked sources.

Deep learning models such as **BERT-based classifiers** can identify potentially misleading content.

6. Graph Analytics

Social media can be represented as a graph:

- Users → Nodes.
- Followers/interactions → Edges.

Graph algorithms can identify suspicious clusters, bot networks, coordinated accounts, and abnormal information spreading.

7. Storage

- **HDFS/Data Lake** stores historical data.
- **NoSQL databases** store user and event information.
- Historical data can be used to retrain ML models.

8. Real-Time Response

When suspicious activity is detected:

- The content can be flagged.
- The account can be temporarily restricted.
- Moderators can receive alerts.
- Additional verification can be requested.

Conclusion

A combination of **Kafka, Flink/Spark Streaming, machine learning, NLP, and graph analytics** provides scalable fraud detection. Stream processing enables rapid detection, while historical Big Data improves the accuracy of machine-learning models.

3.Government Data Platform Problem

It combines census, economic, and geospatial datasets. Different departments need access to shared and integrated data. The system must ensure interoperability across multiple agencies. Strict privacy and security requirements must be followed. Propose a governance and data management strategy.

Introduction

Government departments generate census, economic, healthcare, transportation, taxation, and geospatial data. Since different agencies need to share information, a **centralized governance and integrated Big Data platform** is required.

Proposed Architecture

Government Departments → Data Integration → Data Lake → Processing → Governance & Security → Shared Data Platform → Departments

1. Data Sources

- Census databases.
- Economic and financial datasets.
- Taxation information.
- Transportation data.
- Geospatial/GIS datasets.
- Public-service databases.

2. Data Integration

Tools such as **Apache NiFi, Kafka, and ETL pipelines** can collect and integrate data from different departments.

Common data formats such as:

- JSON
- XML
- CSV
- APIs

- Standard geospatial formats

  can improve interoperability.

  3. Central Data Lake

  A government data lake can store structured, semi-structured, and unstructured data.

  HDFS or cloud object storage can be used for large-scale storage.

  4. Metadata Management

  A common metadata catalog should record:

- Dataset name.
- Data owner.
- Source.
- Data format.
- Update frequency.
- Data quality.
- Access restrictions.

  This helps departments understand and correctly use shared datasets.

  5. Data Governance

  A governance framework should define:

- Data ownership.
- Data stewardship.
- Data quality standards.
- Data retention policies.
- Data sharing rules.
- Data classification.

  6. Interoperability

  All agencies should follow common:

- Data schemas.
- APIs.
- Naming conventions.
- Metadata standards.
- Data exchange formats.

  This allows different government systems to communicate effectively.

  7. Privacy and Security

  Strict security mechanisms should include:

- Role-Based Access Control (RBAC).

- Encryption during storage and transmission.
- Multi-factor authentication.
- Audit logs.
- Data masking and anonymization.
- Secure APIs.

  Sensitive personal information should only be accessible to authorized personnel.

  8. Data Quality

  Automated checks should identify:

- Missing values.
- Duplicate records.
- Incorrect formats.
- Inconsistent information.

  Data quality dashboards can monitor the overall quality of government datasets.

  Conclusion

  A government Big Data platform should combine **data integration, centralized storage, metadata management, interoperability standards, data governance, privacy, and security**. This enables departments to securely share reliable information while protecting citizens' data.

4.Banking Fraud Detection Problem

A bank processes millions of financial transactions every minute. It needs to detect fraudulent activities in real time. The system must analyze historical and streaming transaction data.

Low latency and high accuracy are critical requirements. Design a big data pipeline for fraud detection. Explain the technologies and algorithms you would implement.

Introduction

Banks process millions of transactions continuously. Fraud detection must therefore analyze both **historical and real-time transaction data** with very low latency.

Proposed Big Data Pipeline

Transactions → Kafka → Stream Processing → Feature Engineering → ML Fraud Model → Fraud Score → Alert/Block → Storage

1. Data Sources

The system collects:

- ATM transactions.
- Credit/debit card transactions.

- Online banking transactions.
- Mobile banking transactions.
- Login information.
- Customer transaction history.
- Device and location information.

2. Real-Time Data Ingestion

Apache Kafka can receive millions of transaction events and distribute them across multiple partitions.

This provides:

- High throughput.
- Fault tolerance.
- Scalability.
- Real-time event processing.

3. Stream Processing

Apache Flink or Spark Structured Streaming analyzes transactions immediately.

It can detect:

- Multiple transactions within seconds.
- Unusual transaction locations.
- Abnormally large payments.
- Repeated failed transactions.
- Sudden changes in spending behavior.

4. Feature Engineering

Important features include:

- Transaction amount.
- Transaction frequency.
- Time of transaction.
- Customer spending pattern.
- Location.
- Device information.
- Merchant category.
- Distance between consecutive transactions.

5. Machine Learning Models

A combination of models can improve detection.

Supervised learning:

- Logistic Regression.

- Random Forest.
- XGBoost.
- Neural Networks.

Unsupervised learning:

- Isolation Forest.
- Clustering.
- Anomaly detection.

Supervised models identify known fraud patterns, while anomaly detection can identify previously unseen behavior.

6. Real-Time Fraud Score

Each transaction receives a fraud probability or risk score.

For example:

- **Low risk:** Transaction approved.
- **Medium risk:** Additional authentication required.
- **High risk:** Transaction blocked and security team alerted.

7. Historical Data Storage

Historical transactions can be stored in:

- HDFS/Data Lake.
- Hive.
- Cloud storage.
- NoSQL databases.

Historical data is used to train and update fraud detection models.

8. Model Monitoring

The system should continuously monitor:

- False positives.
- False negatives.
- Detection accuracy.
- Processing latency.
- Model performance.

Models can be retrained as fraud patterns change.

Conclusion

A combination of **Kafka, Flink/Spark Streaming, scalable storage, feature engineering, and machine learning** can provide low-latency fraud detection. Real-time analytics enables immediate action, while historical analytics improves model accuracy.

5.Agriculture Smart Farming Problem

A smart farming system collects soil, weather, and crop data. Sensors and drones provide real-time field information. Farmers want insights to improve yield and resource usage. Data is generated from multiple distributed sources. Design a big data solution for precision agriculture. Explain how analytics can support decision-making.

Introduction

Precision agriculture uses sensors, drones, weather systems, and farm equipment to continuously collect information. Big Data analytics can transform this data into useful decisions for improving **crop yield, irrigation, fertilizer usage, and resource management**.

Proposed Architecture

Sensors/Drones/Weather → IoT Gateway → Kafka → Data Lake → Spark/Flink → ML Analytics → Farmer Dashboard

1. Data Sources

Data can be collected from:

- Soil moisture sensors.
- Temperature sensors.
- Humidity sensors.
- Weather stations.
- Drones.
- Satellite images.
- Crop monitoring devices.
- Farm machinery.

2. IoT Data Collection

Sensors continuously generate data. An **IoT gateway** collects the information from distributed farm locations.

MQTT can be used for lightweight IoT communication.

3. Data Ingestion

Apache Kafka can collect real-time sensor data and distribute it to processing systems.

This is useful because large farms may contain thousands of sensors.

4. Data Storage

A **data lake using HDFS or cloud object storage** can store:

- Sensor readings.
- Historical crop information.
- Drone images.
- Weather records.
- Satellite images.

5. Data Processing

Apache Spark can process large historical datasets.

Spark Streaming or Apache Flink can analyze real-time sensor information.

For example, if soil moisture becomes too low, the system can immediately generate an irrigation alert.

6. Analytics and Machine Learning

Crop Yield Prediction

- Regression models.
- Random Forest.
- Gradient Boosting.
- Neural Networks.

These models can estimate expected crop yield using soil, weather, and crop conditions.

Irrigation Optimization

- Soil moisture and weather data determine when irrigation is required.
- Predictive models can reduce unnecessary water usage.

Crop Disease Detection

- Drone images can be analyzed using **CNN/deep learning models**.
- Diseased areas can be identified at an early stage.

7. Resource Optimization

Analytics can help farmers decide:

- When to irrigate.

- How much water to use.
- Where fertilizer is required.
- When to apply pesticides.
- Which areas require additional monitoring.

8. Farmer Dashboard

A dashboard can display:

- Soil moisture.
- Weather conditions.
- Crop health.
- Predicted yield.
- Irrigation recommendations.
- Disease alerts.

9. Benefits

The system can:

- Increase crop yield.
- Reduce water consumption.
- Reduce fertilizer wastage.
- Detect diseases early.
- Reduce operational costs.
- Support data-driven farming decisions.

Conclusion

A Big Data architecture using **IoT sensors, Kafka, cloud/data-lake storage, Spark/Flink, machine learning, and visualization tools** enables precision agriculture. Real-time and predictive analytics help farmers improve productivity while using water, fertilizer, and other resources efficiently.